Wyklad 3

URL-e, Widoki, QuerySet & Logowanie

Etap 2 — Model studenta i routing

2.1 URL z parametrem: `<int:pk>`

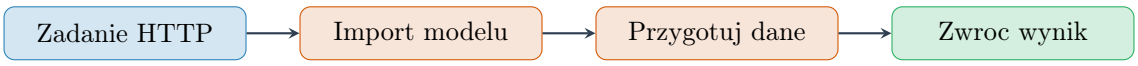
```
path("student/<int:pk>/", views.StudentDetail, name="student-
    detail")
```

Jak dziala `<int:pk>`?

- Django oczekuje **liczby calkowitej** w tym miejscu URL-a
- Wyciaga ja z URL-a i **przekazuje do widoku jako argument** `pk`
- `pk` (*primary key*) to po prostu **id obiektu w bazie danych**
- Jesli w URL-u pojawi sie cos innego niz int — Django zwroci **404**

2.2 Widoki

Widok laczy sie z baza danych i zwraca wynik. Ogolny schemat dzialania:



select_related()

```
# Zamiast osobnego zapytania SQL dla kazdego kursu:
enrollments = Enrollment.objects.select_related("course")

# Pozniej w kodzie mozna odwolac sie bezposrednio:
print(enrollment.course.code)    # brak dodatkowego SQL!

# Pobieranie kursow wykladowcy z sortowaniem:
courses = lecturer.courses.order_by("code")
```

Czym jest `select_related()`?

- Laduje **powiazane obiekty z bazy w jednym zapytaniu SQL** (JOIN zamiast N+1 zapytan)
- Uzywa sie dla relacji **ForeignKey** i **OneToOne**
- Dzieki temu mozna pozniej pisac `enrollment.course.code` bez dodatkowego hitu do bazy
- Argument to **nazwa pola relacji**, np. `select_related("course")`

Podwojny podkreslnik __ w Django ORM

```
# Przejscie przez relacje (traversal)
Enrollment.objects.filter(course__code="CS101")
# "course" to pole ForeignKey, "code" to pole w modelu Course

# Mozna laczyc wiele poziomow:
Enrollment.objects.filter(course__lecturer__last_name="Kowalski")
```

Podwojny podkreslnik `__` oznacza **przejscie po relacji** — Django zamienia to automatycznie na odpowiedni `JOIN` w SQL. Dziala rowniez dla lookupow: `__contains`, `__gte`, `__isnull` itp.

2.3 Szablony i QuerySet

Zasada szablonow

Szablony **nie zajmuja sie logika** — sluza tylko do prezentacji danych. Cala logika biznesowa powinna byc w widokach lub modelach.

QuerySet — co to jest?

QuerySet w Django ORM

- Specyficzna klasa Django ORM reprezentujaca **zbior obiektow z bazy danych** z okreslonymi warunkami
- Podobny do listy, ale z **dodatkowymi funkcjami**
- Odpowiednik w SQLAlchemy: **Query**; w innych ORM podobne koncepty

Sposoby wywolania QuerySet (kiedy faktycznie idzie zapytanie do SQL):

```
qs = Course.objects.filter(active=True)  # jeszcze brak SQL

# Wywolanie nastepuje przy:
for c in qs: ...            # iteracja
qs[0:5]                     # wycinanie (slicing)
qs.first()                  # metoda .first()
qs.last()                   # metoda .last()
qs.count()                  # metoda .count()
list(qs)                    # jawna konwersja
```

3 kluczowe wlasnosci QuerySet

1. **Leniwe wykonanie (lazy evaluation)** — zapytanie SQL jest wysylane *dopiero gdy faktycznie potrzebujemy danych*, nie w momencie budowania QuerySet
2. **Skladanie krokowe** — mozna laczyc wiele operacji filtrowania/sortowania, a i tak powstaje **jedno zapytanie SQL**
3. **Integracja z szablonami** — QuerySet mozna przekazac bezposrednio do szablonu Django i iterowac po nim

Dzieki wczesnemu `select_related` mozna w szablonie odwolac sie do:

```
{% for e in enrollments %}
    {{ e.course.code }}   {# brak dodatkowego SQL - juz zaladowane
        #}
{% endfor %}
```

Uruchomienie serwera deweloperskiego:

```
python serwer.py runserver
```

Etap 3 — Logowanie: wbudowane widoki auth

3.1 Rozszerzamy enroll/urls.py

Django dostarcza **gotowe widoki logowania i wylogowania** — nie trzeba ich pisac od zera.

```
from django.contrib.auth import views as auth_views

urlpatterns += [
    path(
        "login/",
        auth_views.LoginView.as_view(template_name="enroll/login.
            html"),
        name="login"
    ),
    path(
        "logout/",
        auth_views.LogoutView.as_view(),
        name="logout"
    ),
]
```

Widok	Co robi?
LoginView	Wbudowany widok Django. Wyswietla formularz logowania (GET), przetwarza dane (POST), uwierzytelnia uzytkownika i zapisuje sesje. Mozna podac wlasny `template_name`.
LogoutView	Wbudowany widok Django. Niszczy sesje uzytkownika i wylogowuje go. Domyslnie przekierowuje na strone glowna lub `LOGIN_REDIRECT_URL`.

Dlaczego uzywamy wbudowanych widokow?

- Juz zawieraja **cala logike** walidacji, uwierzytelniania, sesji i ochrony CSRF
- Mozna je **dostosowac** przez podanie wlasnego szablonu (`template_name`)
- Sa **przetestowane i bezpieczne** — nie trzeba wymyslac kola na nowo
- Dziedzicza po widokach klasowych (`View`), wiec mozna je dalej rozszerzac

Metoda `as_view()`

```
# as_view() zamienia klase widoku na funkcje,
# ktora Django moze wywolac dla danego URL-a
auth_views.LoginView.as_view(template_name="enroll/login.html")
```

`as_view()` to metoda klasy widoku, ktora **tworzy callable** (obiekt wywolywalny) z klasy. Django w `urlpatterns` oczekuje funkcji, a nie klasy — `as_view()` sluzy wlasnie do tego przejscia.